INFORME TÉCNICO DE RESOLUCIÓN CTF

Captura de un login HTTP

Wireshark · Análisis de tráfico local

Integrante: Felipe Segura

Asignatura: Ciberseguridad Avanzada

Docente: Rodrigo Ledezma

Fecha: 3 de noviembre de 2026

Resultado documentado

Se identificaron las direcciones, el filtro del login y las credenciales admin / admin123 en una captura real.

Elaborado a partir de la resolución y las evidencias disponibles del desafío.

2. Introducción

La misión consiste en capturar un inicio de sesión mediante HTTP y analizar los datos transmitidos. Se debe identificar origen y destino, aislar el login y recuperar credenciales ficticias.

3. Análisis inicial

Se utilizó el servidor HTTP local proporcionado. Como cliente y servidor funcionan en el mismo equipo, la captura se realizó sobre loopback. La hipótesis fue que el cuerpo del POST contendría las credenciales en texto legible al no emplear TLS.

4. Herramientas utilizadas

Servidor y cliente Python: generaron un inicio de sesión reproducible en 127.0.0.1:8090 con datos de prueba.

dumpcap: capturó el tráfico loopback y lo guardó en login_http.pcapng; forma parte de Wireshark.

TShark / Wireshark: decodificaron HTTP, filtraron el POST y reconstruyeron el flujo TCP para leer el formulario.

5. Metodología de resolución

Paso 1. Generación y captura del tráfico

La resolución utilizó ChatGPT (Codex) para automatizar la prueba con resolver_desafio.py. El script inició el servidor, capturó durante 12 segundos y envió un POST /login mediante un cliente Python. El comando de captura registrado en el script es:

```
dumpcap -i \Device\NPF_Loopback -f "tcp port 8090" -a duration:12 -w
    evidencias/login_http.pcapng
```

Paso 2. Filtrado y decodificación

```
tshark -n -r evidencias/login_http.pcapng -d tcp.port==8090,http -Y 'http.request.method ==
    "POST"' -V
```

-r lee la captura, -d fuerza la decodificación HTTP del puerto 8090, -Y aplica el filtro y -V muestra detalles. En la interfaz gráfica, si HTTP no se reconoce, se puede usar Decode As para el mismo puerto. El filtro documentado es:

```
tcp.port == 8090 && http.request.method == "POST"
```

Paso 3. Lectura del formulario y del flujo

```
tshark -n -r evidencias/login_http.pcapng -d tcp.port==8090,http -q -z follow,tcp,ascii,1
```

El POST se identifica en el paquete 20 tras el reensamblado de los segmentos 18 y 20. El flujo TCP 1 contiene el formulario y una respuesta HTTP 200 con «Acceso concedido». El puerto de origen es 50600 y el de destino 8090.

6. Evidencias

Extracto del flujo TCP conservado en evidencias/flujo_login.txt. Es una transcripción de evidencia real, no una captura de la interfaz gráfica:

```
Node 0: 127.0.0.1:50600
Node 1: 127.0.0.1:8090

POST /login HTTP/1.1
Host: 127.0.0.1:8090
Content-Type: application/x-www-form-urlencoded

usuario=admin&password=admin123

HTTP/1.0 200 OK
Acceso concedido
```

Respaldo: login_http.pcapng, detalle_post.txt y login_post.json. La imagen Desafio.png contiene el enunciado y un ejemplo ilustrativo, por lo que no se presenta como evidencia de esta ejecución. No hay una captura propia de la interfaz gráfica entre los archivos disponibles.

7. Uso de pistas

Se siguió la misión del enunciado: localizar el login HTTP y revelar datos inventados. No se registran pistas adicionales. Codex ayudó a automatizar la captura y analizarla con herramientas de Wireshark.

8. Obtención de la FLAG o resultado

Este desafío no solicita una FLAG con formato especial. Los resultados son:

```
IP origen:  127.0.0.1
IP destino: 127.0.0.1
Puerto destino: 8090
Login: POST /login, paquete 20
Usuario: admin
Contraseña: admin123
```

Las IP coinciden porque la prueba se hizo en el mismo equipo. Las credenciales están respaldadas por el cuerpo del POST y el flujo TCP.

9. Análisis del proceso

La decisión clave fue capturar la interfaz loopback y decodificar el puerto 8090 como HTTP. El seguimiento del flujo permitió leer el formulario completo. La actividad se ejecutó con un cliente Python, como registra la resolución; no se atribuye la captura a un login realizado manualmente en el navegador.

10. Análisis ético

Se capturó únicamente tráfico propio del servidor local con credenciales ficticias. En otros entornos, una captura puede exponer datos privados y requiere autorización. HTTPS protege el contenido durante el transporte mediante TLS.

11. Conclusiones

La captura permitió identificar las IP y recuperar las credenciales del login. El reto demostró que HTTP expone el contenido del formulario y que HTTPS es necesario para protegerlo durante el transporte.

Fuentes y archivos de respaldo

Pauta_Resolucion_Desafio_CTF.docx y Desafio Wireshark/Desafio.png: pauta y misión.

resolucion.md y resolver_desafio.py: procedimiento automatizado.

evidencias/login_http.pcapng, flujo_login.txt y detalle_post.txt: captura, flujo y paquete analizado.